UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA DE SOFTWARE

ANÁLISIS Y DISEÑO DE SOFTWARE

Unidad 1 – Guía 2 (U1T2)

Casos de uso, clases de análisis y diagramas de secuencia de análisis

Proyecto: AgendaMed Renacer v4.0 – Propuesta 1
Caso CRUD: Gestionar Sucursales (RF-CFG-01)

Integrantes:
Esteban Santos
Brayan Sebastian Criollo Vega
Maycol Celi

Docente: MSc. Jenny Alexandra Ruiz Robalino
NRC: 36839

Sangolquí, 20 de octubre de 2026

3. Caso base actualizado: CRUD de Sucursales

Para esta segunda entrega tomamos como caso base el CRUD de **Gestionar Sucursales** (RF-CFG-01), que pertenece al módulo Configurar Clínica (RF-N0-02) del trabajo anterior. Lo elegimos porque es un catálogo base del sistema, maneja pocos datos y sus reglas están bien definidas en el SRS, además de que sus operaciones calzan directamente con el esquema Agregar, Actualizar, Eliminar y Mostrar Todo que plantea la guía.

La diferencia con el ejemplo del estudiante está en la eliminación. En AgendaMed Renacer una sucursal nunca se borra físicamente: solo se inactiva (baja lógica), y el sistema únicamente lo permite cuando la sucursal no tiene citas futuras en estado PENDIENTE_PAGO o CONFIRMADA. Por eso, en este trabajo la operación «Eliminar» se modela como «Inactivar sucursal».

La interfaz que tomamos como referencia es la pantalla «Gestión de Sucursales», a la que accede únicamente el Administrador. Tiene cuatro campos de entrada, una tabla con las sucursales registradas y cuatro botones de acción. Las sucursales iniciales son Matriz Sur, La Arcadia y Sangolquí (esta última con atención walk-in).

Datos de la sucursal

Dato	Tipo sugerido	Descripción	Validación mínima
ID	Entero autogenerado	Identificador interno de la sucursal; lo asigna el sistema.	No editable y único.
Nombre	Texto	Nombre de la sucursal mostrado en el formulario, la tabla y la búsqueda de disponibilidad.	Obligatorio, único, entre 3 y 50 caracteres.
Dirección	Texto	Dirección física de la sucursal.	Obligatoria; no debe contener solo espacios.
Teléfono	Texto numérico	Teléfono de contacto de la sucursal.	Obligatorio, 9 o 10 dígitos numéricos que inician en 0.
Atiende walk-in	Lógico (Sí / No)	Indicador que habilita la sucursal para agendar pacientes sin cita previa (RF-CIT-04).	Obligatorio.
Estado	Texto (ACTIVA / INACTIVA)	Estado de la sucursal; lo gestiona el sistema, no se digita.	Toda sucursal nueva nace ACTIVA; una INACTIVA no se ofrece en RF-CIT-01.

Botones de la interfaz

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Registrar	Registrar sucursal	Crea una nueva sucursal luego de validar nombre, dirección, teléfono e indicador walk-in.
Modificar	Modificar sucursal	Actualiza los datos de una sucursal existente identificada por su ID.
Inactivar	Inactivar sucursal	Da de baja lógica a la sucursal seleccionada, siempre que no tenga citas futuras.
Mostrar Todas	Consultar sucursales	Recupera y presenta la lista completa de sucursales registradas, activas e inactivas.

4. Requisitos funcionales de nivel 0

Estos requisitos salen de bajar un nivel el RF-CFG-01 del SRS, que a su vez corresponde al RF-N0-02 Configurar Clínica de la guía anterior. Conservamos el identificador original del SRS y le agregamos un sufijo por cada operación (RF-CFG-01.1 a RF-CFG-01.6), de modo que la trazabilidad hacia el documento anterior se mantiene directa.

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-CFG-01.1	El sistema debe permitir registrar una sucursal con nombre, dirección, teléfono e indicador «Atiende walk-in».	Administrador	Registrar sucursal
RF-CFG-01.2	El sistema debe permitir modificar los datos de una sucursal existente.	Administrador	Modificar sucursal
RF-CFG-01.3	El sistema debe permitir inactivar una sucursal existente (baja lógica) cuando corresponda.	Administrador	Inactivar sucursal
RF-CFG-01.4	El sistema debe permitir mostrar todas las sucursales registradas.	Administrador	Consultar sucursales
RF-CFG-01.5	El sistema debe validar los datos obligatorios (nombre único de 3 a 50 caracteres, dirección, teléfono de 9 o 10 dígitos que inicia en 0) antes de registrar o modificar.	Sistema	Validar datos de la sucursal
RF-CFG-01.6	El sistema debe verificar que la sucursal no tenga citas futuras en estado PENDIENTE_PAGO o CONFIRMADA antes de inactivarla.	Sistema	Verificar citas futuras

Cada operación deja registro en la bitácora de auditoría (RNF-13) y se ejecuta como transacción atómica (RNF-06), por lo que ambos requisitos no funcionales se tienen presentes en el modelo de clases y en las secuencias.

5. Diagrama de casos de uso de análisis

El diagrama representa las funciones visibles para el Administrador, único actor del módulo. Las validaciones no las ejecuta él directamente, sino que son comportamiento obligatorio del sistema, por lo que se modelan como casos de uso incluidos: Registrar y Modificar siempre validan los datos, e Inactivar siempre verifica las citas futuras. Consultar sucursales no incluye nada porque solo recupera información.

Figura 1. Casos de uso de análisis - Gestionar Sucursales (RF-CFG-01)

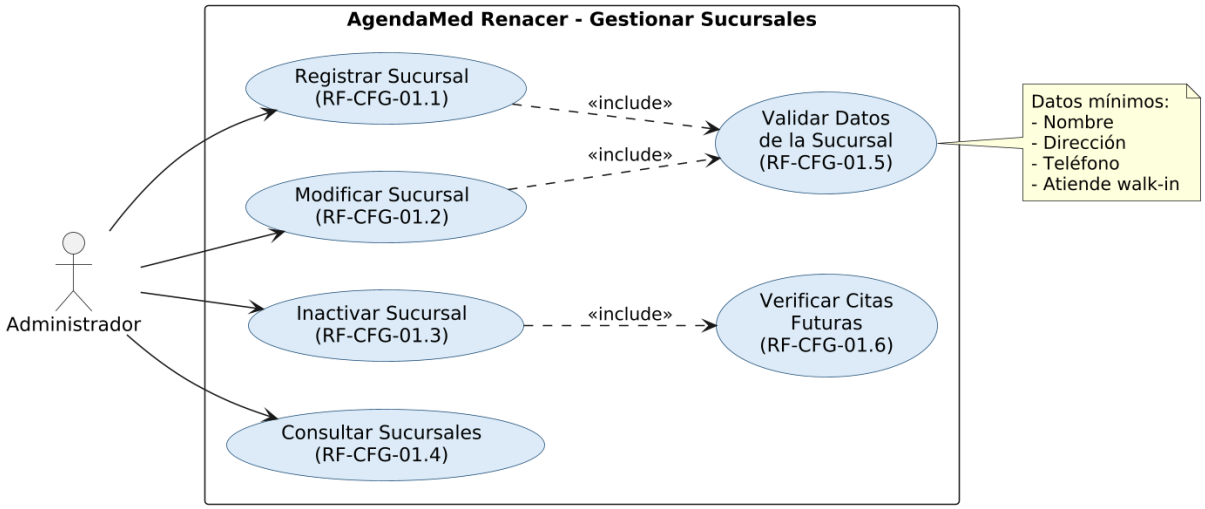


Figura 1. Casos de uso de análisis – Gestionar Sucursales (RF-CFG-01).

Código PlantUML

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false
skinparam dpi 150
skinparam defaultFontName Arial
skinparam usecase {
  BackgroundColor #DCEBF7
  BorderColor #1F4E79
}
title Figura 1. Casos de uso de análisis - Gestionar Sucursales (RF-CFG-01)

actor "Administrador" as Adm

rectangle "AgendaMed Renacer - Gestionar Sucursales" {
  usecase "Registrar Sucursal\n(RF-CFG-01.1)" as CU1
  usecase "Modificar Sucursal\n(RF-CFG-01.2)" as CU2
  usecase "Inactivar Sucursal\n(RF-CFG-01.3)" as CU3
  usecase "Consultar Sucursales\n(RF-CFG-01.4)" as CU4
  usecase "Validar Datos\nde la Sucursal\n(RF-CFG-01.5)" as CU5
  usecase "Verificar Citas\nFuturas\n(RF-CFG-01.6)" as CU6
}

Adm --> CU1
Adm --> CU2
Adm --> CU3
Adm --> CU4

CU1 ..> CU5 : <<include>>
CU2 ..> CU5 : <<include>>
CU3 ..> CU6 : <<include>>

note right of CU5
  Datos mínimos:
  - Nombre
  - Dirección
  - Teléfono
  - Atiende walk-in
end note
@enduml
```

Justificación de las relaciones «include»

Tipo	Caso de uso base	Caso de uso incluido	Justificación funcional
«include»	Registrar sucursal (RF-CFG-01.1)	Validar datos de la sucursal (RF-CFG-01.5)	No se puede crear una sucursal con nombre repetido, teléfono mal formado o campos vacíos; la validación ocurre siempre.
«include»	Modificar sucursal (RF-CFG-01.2)	Validar datos de la sucursal (RF-CFG-01.5)	Se aplican las mismas reglas del registro, por lo que el comportamiento se reutiliza en lugar de duplicarse.
«include»	Inactivar sucursal (RF-CFG-01.3)	Verificar citas futuras (RF-CFG-01.6)	El SRS bloquea la inactivación si existen citas futuras; la verificación es obligatoria en cada intento.

En el diagrama de Nivel 1 de la guía anterior, RF-CFG-01 era una sola elipse sin relaciones. Estos «include» aparecen ahora porque estamos descomponiendo ese requisito en sus operaciones y reglas internas, sin contradecir lo declarado en el SRS.

6. Derivación de clases de análisis

A partir de los casos de uso identificamos las clases de análisis usando los estereotipos boundary, control y entity, además de repositorios para representar la persistencia sin entrar todavía en detalles de base de datos. Se agregan la entidad RegistroAuditoria, que exige RNF-13, y la entidad Cita, que solo se consulta para saber si la sucursal tiene citas futuras.

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el CRUD
Boundary / Frontera	Recibe datos del Administrador y muestra resultados.	FormularioGestionSucursales	Representa la pantalla con los campos Nombre, Dirección, Teléfono, Walk-in y los botones Registrar, Modificar, Inactivar y Mostrar Todas.
Control	Coordina la lógica de cada caso de uso.	ControlSucursal	Valida datos, verifica nombre único y citas futuras, y decide si registra, modifica, inactiva o consulta.
Entity / Entidad	Representa la información del dominio.	Sucursal	Contiene id, nombre, dirección, teléfono, atiendeWalkIn y estado.
Entity / Entidad	Guarda la trazabilidad de cada acción (RNF-13).	RegistroAuditoria	Se crea en cada registro, modificación o inactivación exitosa.
Entity / Entidad	Representa la cita agendada en una sucursal.	Cita	Solo se consulta (estado y fecha) para decidir si se puede inactivar.
Repositorio	Conserva y recupera sucursales.	RepositorioSucursal	Permite guardar, buscar, actualizar, listar y comprobar que el nombre no se repita.
Repositorio	Consulta las citas asociadas a una sucursal.	RepositorioCita	Cuenta las citas futuras PENDIENTE_PAGO o CONFIRMADA de una sucursal.
Repositorio	Conserva el historial de auditoría.	RepositorioAuditoria	Registra cada acción del Administrador sobre las sucursales.

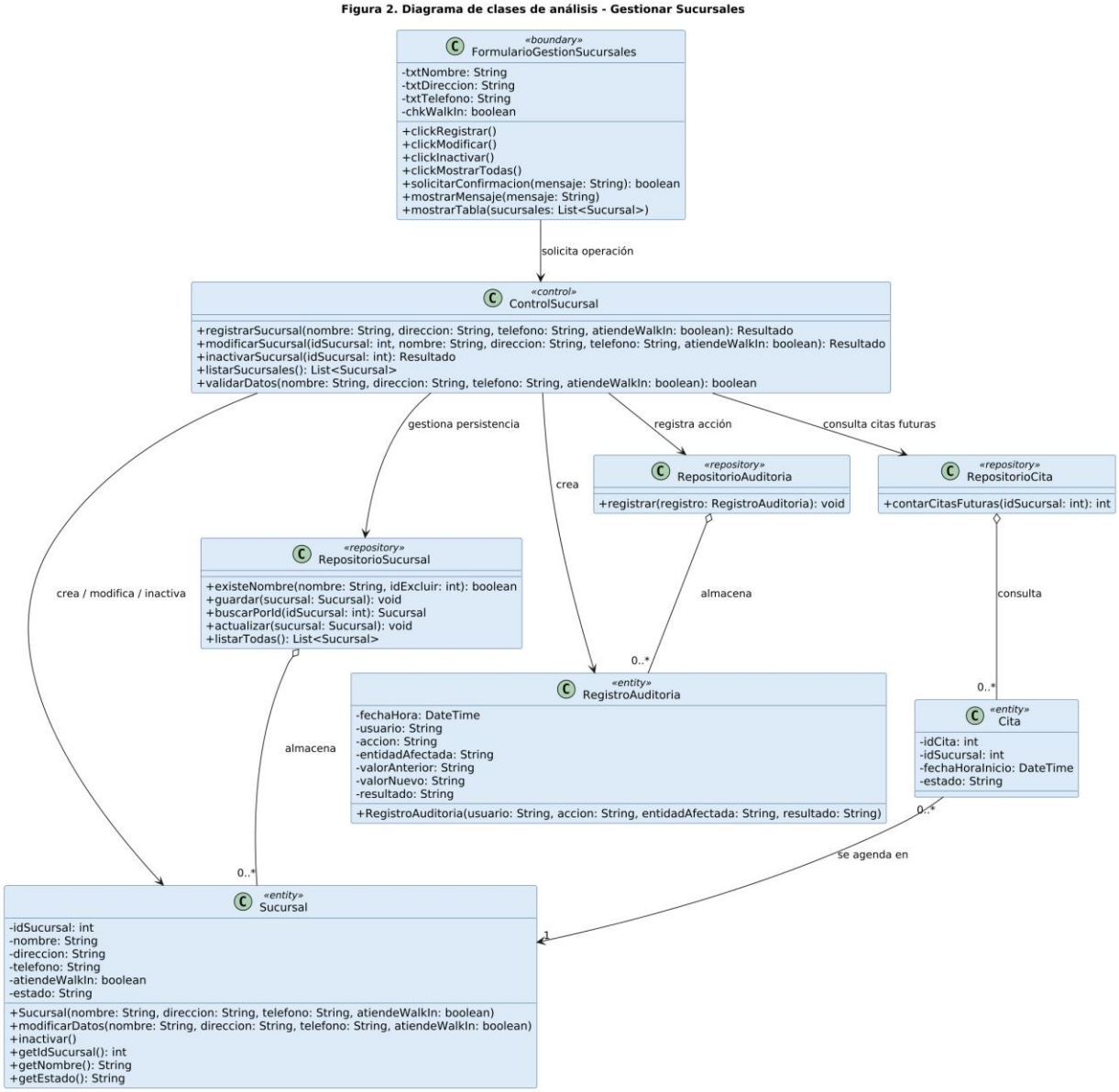


Figura 2. Diagrama de clases de análisis – Gestionar Sucursales.

Código PlantUML del diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam dpi 150
skinparam defaultFontName Arial
skinparam class {
  BackgroundColor #DCEBF7
  BorderColor #1F4E79
}
title Figura 2. Diagrama de clases de análisis - Gestionar Sucursales

class "FormularioGestionSucursales" <<boundary>> {
  - txtNombre: String
  - txtDireccion: String
  - txtTelefono: String
  - chkWalkIn: boolean
  + clickRegistrar()
  + clickModificar()
  + clickInactivar()
  + clickMostrarTodas()
  + solicitarConfirmacion(mensaje: String): boolean
  + mostrarMensaje(mensaje: String)
```

```
  + mostrarTabla(sucursales: List<Sucursal>)
}

class "ControlSucursal" <<control>> {
  + registrarSucursal(nombre: String, direccion: String, telefono: String, atiendeWalkIn: boolean):
Resultado
  + modificarSucursal(idSucursal: int, nombre: String, direccion: String, telefono: String,
atiendeWalkIn: boolean): Resultado
  + inactivarSucursal(idSucursal: int): Resultado
  + listarSucursales(): List<Sucursal>
  + validarDatos(nombre: String, direccion: String, telefono: String, atiendeWalkIn: boolean): boolean
}

class "Sucursal" <<entity>> {
  - idSucursal: int
  - nombre: String
  - direccion: String
  - telefono: String
  - atiendeWalkIn: boolean
  - estado: String
  + Sucursal(nombre: String, direccion: String, telefono: String, atiendeWalkIn: boolean)
  + modificarDatos(nombre: String, direccion: String, telefono: String, atiendeWalkIn: boolean)
  + inactivar()
  + getIdSucursal(): int
  + getNombre(): String
  + getEstado(): String
}

class "Cita" <<entity>> {
  - idCita: int
  - idSucursal: int
  - fechaHoraInicio: DateTime
  - estado: String
}

class "RegistroAuditoria" <<entity>> {
  - fechaHora: DateTime
  - usuario: String
  - accion: String
  - entidadAfectada: String
  - valorAnterior: String
  - valorNuevo: String
  - resultado: String
  + RegistroAuditoria(usuario: String, accion: String, entidadAfectada: String, resultado: String)
}

class "RepositorioSucursal" <<repository>> {
  + existeNombre(nombre: String, idExcluir: int): boolean
  + guardar(sucursal: Sucursal): void
  + buscarPorId(idSucursal: int): Sucursal
  + actualizar(sucursal: Sucursal): void
  + listarTodas(): List<Sucursal>
}

class "RepositorioCita" <<repository>> {
  + contarCitasFuturas(idSucursal: int): int
}

class "RepositorioAuditoria" <<repository>> {
  + registrar(registro: RegistroAuditoria): void
}

FormularioGestionSucursales --> ControlSucursal : solicita operación
ControlSucursal --> Sucursal : crea / modifica / inactiva
ControlSucursal --> RegistroAuditoria : crea
ControlSucursal --> RepositorioSucursal : gestiona persistencia
ControlSucursal --> RepositorioCita : consulta citas futuras
ControlSucursal --> RepositorioAuditoria : registra acción
RepositorioSucursal o-- "0..*" Sucursal : almacena
RepositorioCita o-- "0..*" Cita : consulta
RepositorioAuditoria o-- "0..*" RegistroAuditoria : almacena
Cita "0..*" --> "1" Sucursal : se agenda en
@enduml
```

7. Matriz de relación: casos de uso, clases y secuencias

La primera matriz sigue el esquema de la guía y muestra qué hace cada clase en cada caso de uso. La segunda baja la trazabilidad hasta el requisito, el requisito no funcional asociado, los criterios de aceptación del SRS y la figura donde se encuentra la secuencia.

7.1 Responsabilidades por caso de uso

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Registrar sucursal	Captura nombre, dirección, teléfono y walk-in desde el formulario.	Valida datos, verifica que el nombre sea único y coordina el guardado.	Crea Sucursal y la guarda en RepositorioSucursal; crea RegistroAuditoria y lo guarda en RepositorioAuditoria.	Ingresar datos → validar → verificar nombre → crear entidad → guardar → auditar → confirmar.
Modificar sucursal	Captura el ID de la sucursal seleccionada y los nuevos valores.	Valida datos, busca la sucursal, verifica nombre y coordina la actualización.	Actualiza Sucursal en RepositorioSucursal; registra la acción en RepositorioAuditoria.	Ingresar cambios → validar → buscar → verificar nombre → actualizar → auditar → confirmar.
Inactivar sucursal	Solicita la selección de la fila y la confirmación.	Busca la sucursal, verifica citas futuras y coordina la baja lógica.	RepositorioCita cuenta citas futuras; Sucursal cambia a INACTIVA; se registra la auditoría.	Seleccionar → confirmar → buscar → contar citas → inactivar → auditar → notificar.
Consultar sucursales	Solicita mostrar la lista y presenta la tabla.	Solicita todos los registros.	RepositorioSucursal devuelve la lista de Sucursal.	Solicitar listado → recuperar lista → mostrar tabla.

7.2 Trazabilidad RF – caso de uso – clases – secuencia

RF	Caso de uso	Clases participantes	Secuencia	RNF	Criterios del SRS (RF-CFG-01)
RF-CFG-01.1	Registrar sucursal	FormularioGestionSucursales, ControlSucursal, Sucursal, RegistroAuditoria, RepositorioSucursal, RepositorioAuditoria	Figura 3	RNF-06, RNF-13	CA-01, CA-02, CA-04
RF-CFG-01.2	Modificar sucursal	FormularioGestionSucursales, ControlSucursal, Sucursal, RegistroAuditoria, RepositorioSucursal, RepositorioAuditoria	Figura 4	RNF-06, RNF-13	CA-01, CA-02
RF-CFG-01.3	Inactivar sucursal	FormularioGestionSucursales, ControlSucursal, Sucursal, RegistroAuditoria, RepositorioSucursal, RepositorioCita, RepositorioAuditoria	Figura 5	RNF-06, RNF-13	CA-03
RF-CFG-01.4	Consultar sucursales	FormularioGestionSucursales, ControlSucursal, RepositorioSucursal	Figura 6	RNF-02	Apoya la selección en CA-03 y CA-04
RF-CFG-01.5	Validar datos de la sucursal	ControlSucursal (validarDatos), RepositorioSucursal (existeNombre)	Figuras 3 y 4	RNF-06	CA-01, CA-02
RF-CFG-01.6	Verificar citas futuras	ControlSucursal, RepositorioCita (contarCitasFuturas), Cita	Figura 5	RNF-06	CA-03

8. Diagramas de secuencia de análisis

Elaboramos un diagrama de secuencia por cada caso de uso principal para que ninguno quede sobrecargado. En todos los diagramas el flujo principal es el de éxito, y los flujos alternos relevantes del SRS (datos inválidos, nombre duplicado y sucursal con citas futuras) se muestran con fragmentos «alt». Cada mensaje corresponde a una operación declarada en el diagrama de clases de la sección 6.

8.1 Secuencia: Registrar sucursal

Figura 3. Secuencia de análisis - Registrar sucursal

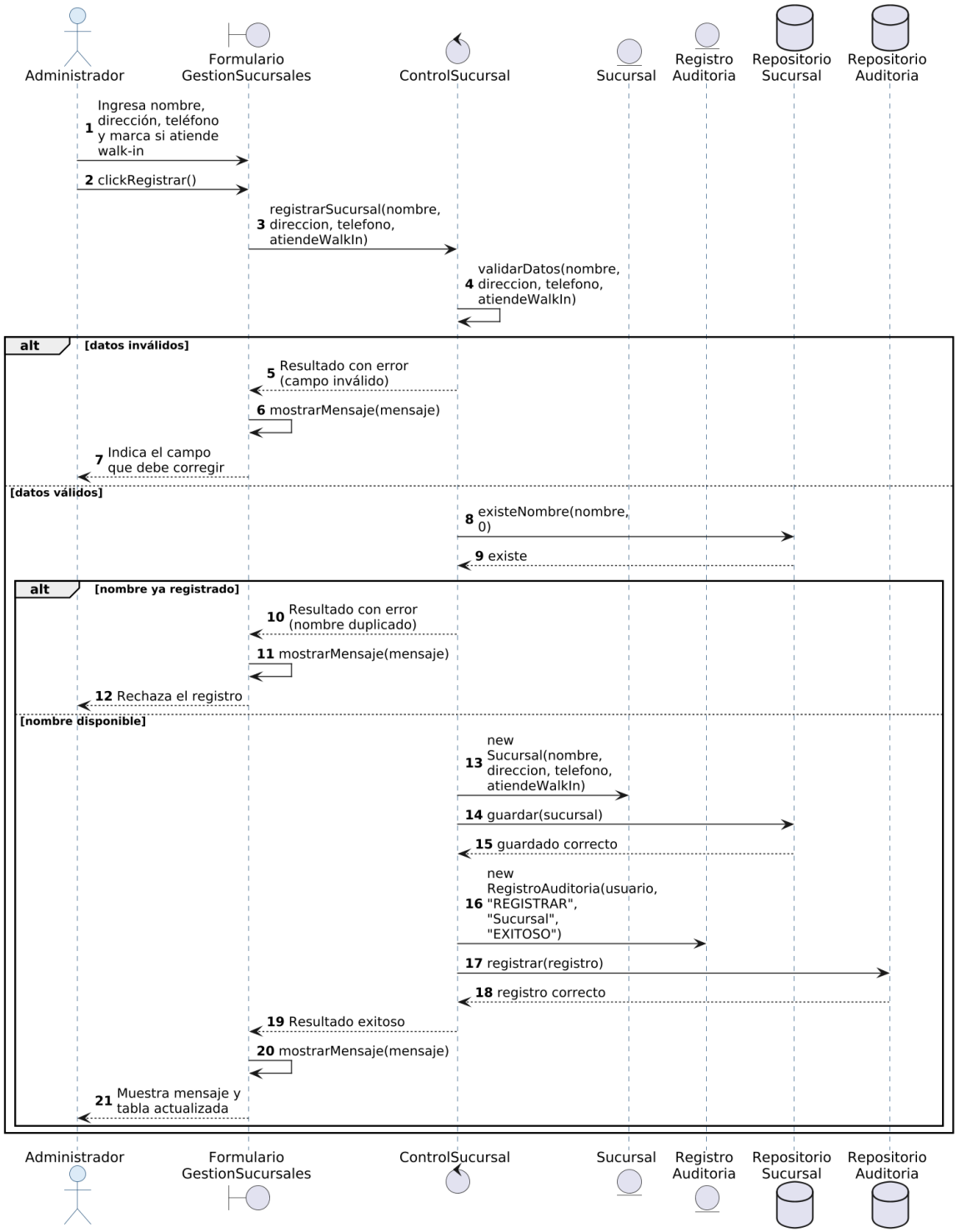


Figura 3. Secuencia de análisis – Registrar sucursal.

Código PlantUML

```
@startuml
skinparam shadowing false
skinparam dpi 150
skinparam maxMessageSize 130
```

```
skinparam ParticipantPadding 2
skinparam BoxPadding 4
skinparam defaultFontName Arial
skinparam sequence {
  ParticipantBackgroundColor #DCEBF7
  ParticipantBorderColor #1F4E79
  ActorBackgroundColor #DCEBF7
  ActorBorderColor #1F4E79
  LifeLineBorderColor #1F4E79
}
title Figura 3. Secuencia de análisis - Registrar sucursal
autonumber

actor "Administrador" as Adm
boundary "Formulario\nGestionSucursales" as UI
control "ControlSucursal" as Ctrl
entity "Sucursal" as Suc
entity "Registro\nAuditoria" as Aud
database "Repositorio\nSucursal" as RepoS
database "Repositorio\nAuditoria" as RepoA

Adm -> UI : Ingresa nombre, dirección, teléfono\ny marca si atiende walk-in
Adm -> UI : clickRegistrar()
UI -> Ctrl : registrarSucursal(nombre, direccion, telefono, atiendeWalkIn)
Ctrl -> Ctrl : validarDatos(nombre, direccion, telefono, atiendeWalkIn)
alt datos inválidos
  Ctrl --> UI : Resultado con error (campo inválido)
  UI -> UI : mostrarMensaje(mensaje)
  UI --> Adm : Indica el campo que debe corregir
else datos válidos
  Ctrl -> RepoS : existeNombre(nombre, 0)
  RepoS --> Ctrl : existe
  alt nombre ya registrado
    Ctrl --> UI : Resultado con error (nombre duplicado)
    UI -> UI : mostrarMensaje(mensaje)
    UI --> Adm : Rechaza el registro
  else nombre disponible
    Ctrl -> Suc : new Sucursal(nombre, direccion, telefono, atiendeWalkIn)
    Ctrl -> RepoS : guardar(sucursal)
    RepoS --> Ctrl : guardado correcto
    Ctrl -> Aud : new RegistroAuditoria(usuario, "REGISTRAR", "Sucursal", "EXITOSO")
    Ctrl -> RepoA : registrar(registro)
    RepoA --> Ctrl : registro correcto
    Ctrl --> UI : Resultado exitoso
    UI -> UI : mostrarMensaje(mensaje)
    UI --> Adm : Muestra mensaje y tabla actualizada
  end
end
@enduml
```

8.2 Secuencia: Modificar sucursal

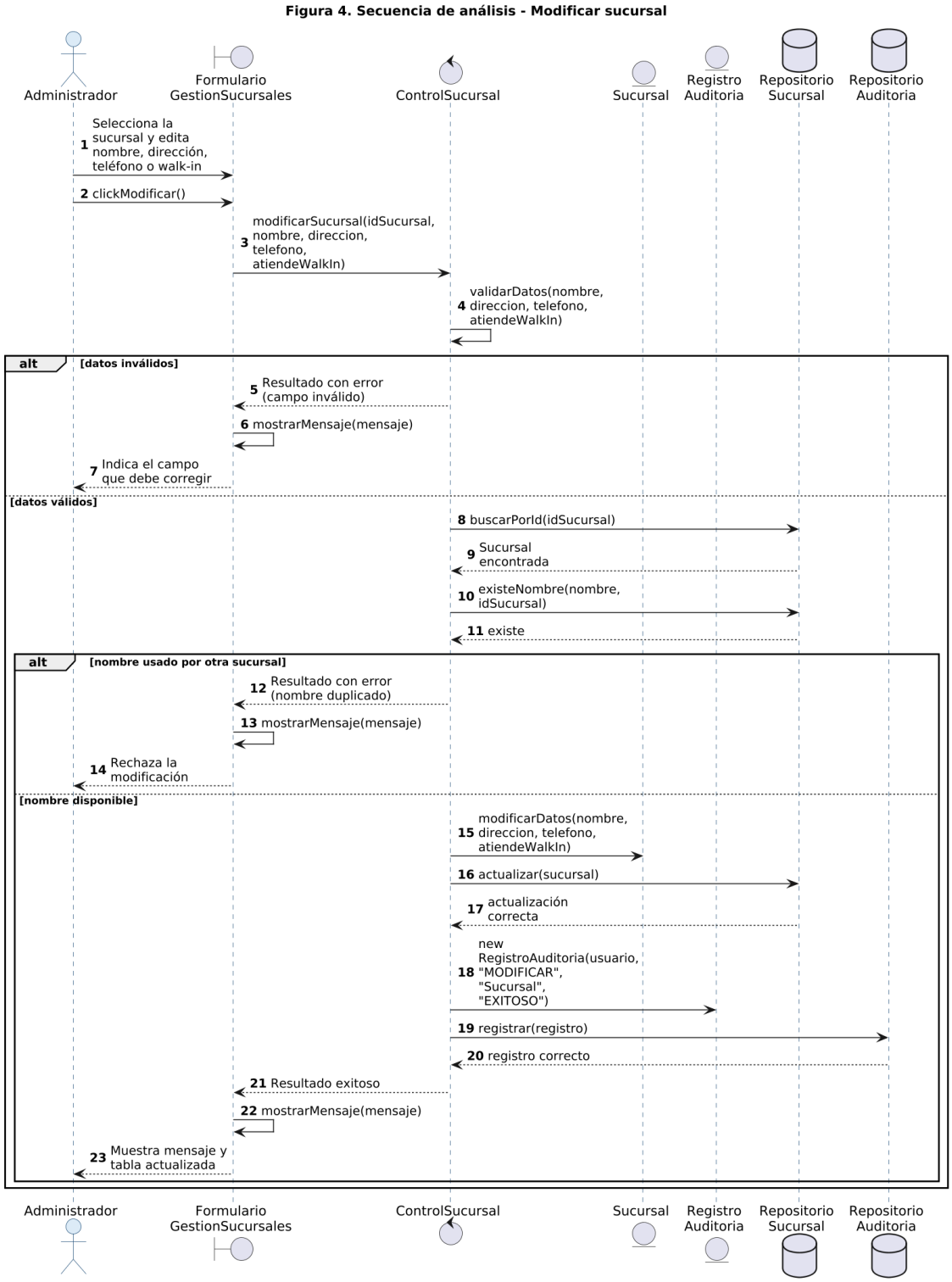


Figura 4. Secuencia de análisis – Modificar sucursal.

Código PlantUML

```
@startuml
skinparam shadowing false
skinparam dpi 150
skinparam maxMessageSize 130
```

```
skinparam ParticipantPadding 2
skinparam BoxPadding 4
skinparam defaultFontName Arial
skinparam sequence {
  ParticipantBackgroundColor #DCEBF7
  ParticipantBorderColor #1F4E79
  ActorBackgroundColor #DCEBF7
  ActorBorderColor #1F4E79
  LifeLineBorderColor #1F4E79
}
title Figura 4. Secuencia de análisis - Modificar sucursal
autonumber

actor "Administrador" as Adm
boundary "Formulario\nGestionSucursales" as UI
control "ControlSucursal" as Ctrl
entity "Sucursal" as Suc
entity "Registro\nAuditoria" as Aud
database "Repositorio\nSucursal" as RepoS
database "Repositorio\nAuditoria" as RepoA

Adm -> UI : Selecciona la sucursal y edita\nnombre, dirección, teléfono o walk-in
Adm -> UI : clickModificar()
UI -> Ctrl : modificarSucursal(idSucursal, nombre, direccion, telefono, atiendeWalkIn)
Ctrl -> Ctrl : validarDatos(nombre, direccion, telefono, atiendeWalkIn)
alt datos inválidos
  Ctrl --> UI : Resultado con error (campo inválido)
  UI -> UI : mostrarMensaje(mensaje)
  UI --> Adm : Indica el campo que debe corregir
else datos válidos
  Ctrl -> RepoS : buscarPorId(idSucursal)
  RepoS --> Ctrl : Sucursal encontrada
  Ctrl -> RepoS : existeNombre(nombre, idSucursal)
  RepoS --> Ctrl : existe
  alt nombre usado por otra sucursal
    Ctrl --> UI : Resultado con error (nombre duplicado)
    UI -> UI : mostrarMensaje(mensaje)
    UI --> Adm : Rechaza la modificación
  else nombre disponible
    Ctrl -> Suc : modificarDatos(nombre, direccion, telefono, atiendeWalkIn)
    Ctrl -> RepoS : actualizar(sucursal)
    RepoS --> Ctrl : actualización correcta
    Ctrl -> Aud : new RegistroAuditoria(usuario, "MODIFICAR", "Sucursal", "EXITOSO")
    Ctrl -> RepoA : registrar(registro)
    RepoA --> Ctrl : registro correcto
    Ctrl --> UI : Resultado exitoso
    UI -> UI : mostrarMensaje(mensaje)
    UI --> Adm : Muestra mensaje y tabla actualizada
  end
end
@enduml
```

8.3 Secuencia: Inactivar sucursal

Figura 5. Secuencia de análisis - Inactivar sucursal

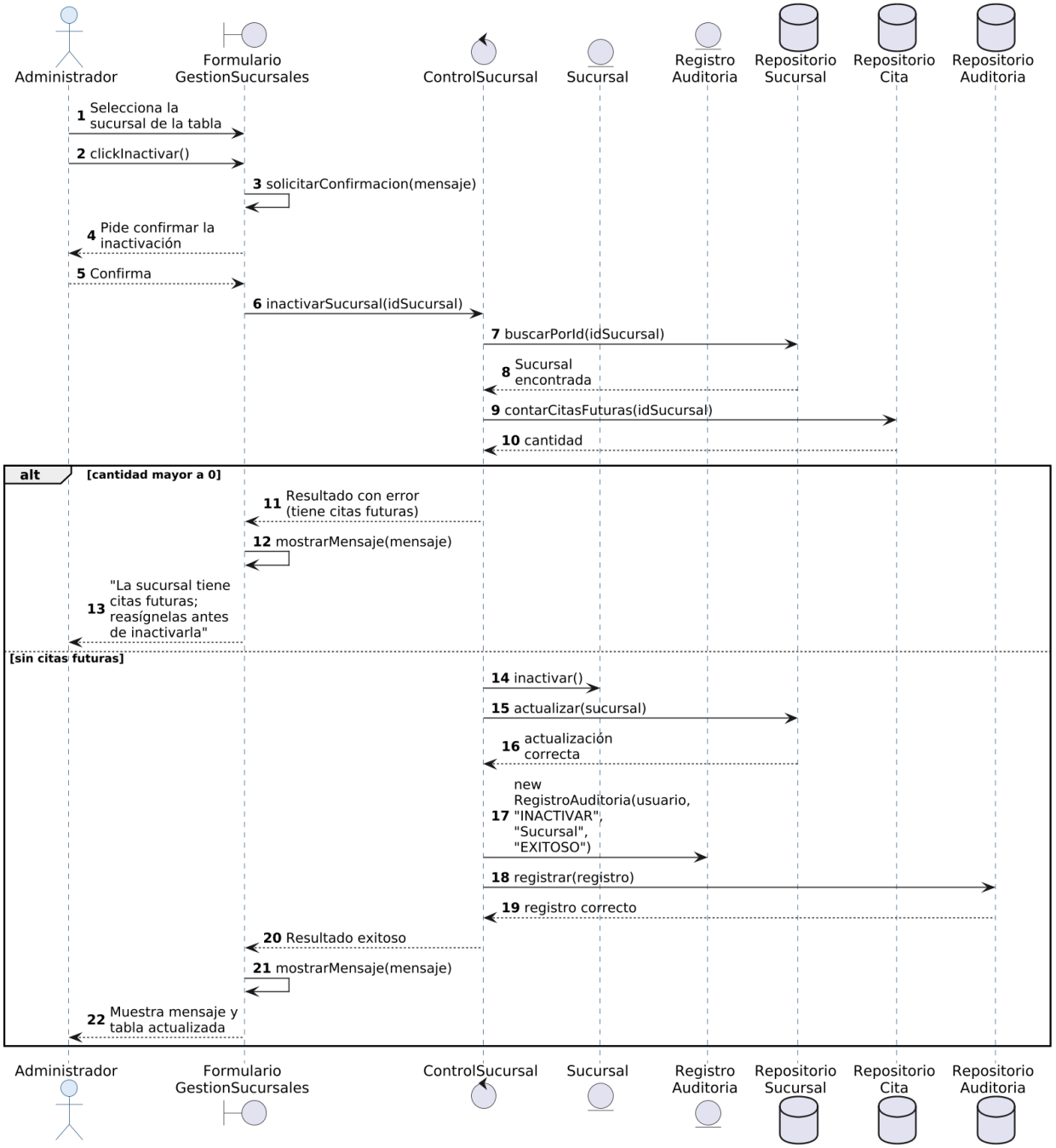


Figura 5. Secuencia de análisis – Inactivar sucursal.

Código PlantUML

```
@startuml
skinparam shadowing false
skinparam dpi 150
skinparam maxMessageSize 130
skinparam ParticipantPadding 2
skinparam BoxPadding 4
skinparam defaultFontName Arial
skinparam sequence {
  ParticipantBackgroundColor #DCEBF7
  ParticipantBorderColor #1F4E79
```

```
  ActorBackgroundColor #DCEBF7
  ActorBorderColor #1F4E79
  LifeLineBorderColor #1F4E79
}
title Figura 5. Secuencia de análisis - Inactivar sucursal
autonumber

actor "Administrador" as Adm
boundary "Formulario\nGestionSucursales" as UI
control "ControlSucursal" as Ctrl
entity "Sucursal" as Suc
entity "Registro\nAuditoria" as Aud
database "Repositorio\nSucursal" as RepoS
database "Repositorio\nCita" as RepoC
database "Repositorio\nAuditoria" as RepoA

Adm -> UI : Selecciona la sucursal de la tabla
Adm -> UI : clickInactivar()
UI -> UI : solicitarConfirmacion(mensaje)
UI --> Adm : Pide confirmar la inactivación
Adm --> UI : Confirma
UI -> Ctrl : inactivarSucursal(idSucursal)
Ctrl -> RepoS : buscarPorId(idSucursal)
RepoS --> Ctrl : Sucursal encontrada
Ctrl -> RepoC : contarCitasFuturas(idSucursal)
RepoC --> Ctrl : cantidad
alt cantidad mayor a 0
  Ctrl --> UI : Resultado con error (tiene citas futuras)
  UI -> UI : mostrarMensaje(mensaje)
  UI --> Adm : "La sucursal tiene citas futuras;\nreasígnelas antes de inactivarla"
else sin citas futuras
  Ctrl -> Suc : inactivar()
  Ctrl -> RepoS : actualizar(sucursal)
  RepoS --> Ctrl : actualización correcta
  Ctrl -> Aud : new RegistroAuditoria(usuario, "INACTIVAR", "Sucursal", "EXITOSO")
  Ctrl -> RepoA : registrar(registro)
  RepoA --> Ctrl : registro correcto
  Ctrl --> UI : Resultado exitoso
  UI -> UI : mostrarMensaje(mensaje)
  UI --> Adm : Muestra mensaje y tabla actualizada
end
@enduml
```

8.4 Secuencia: Consultar sucursales

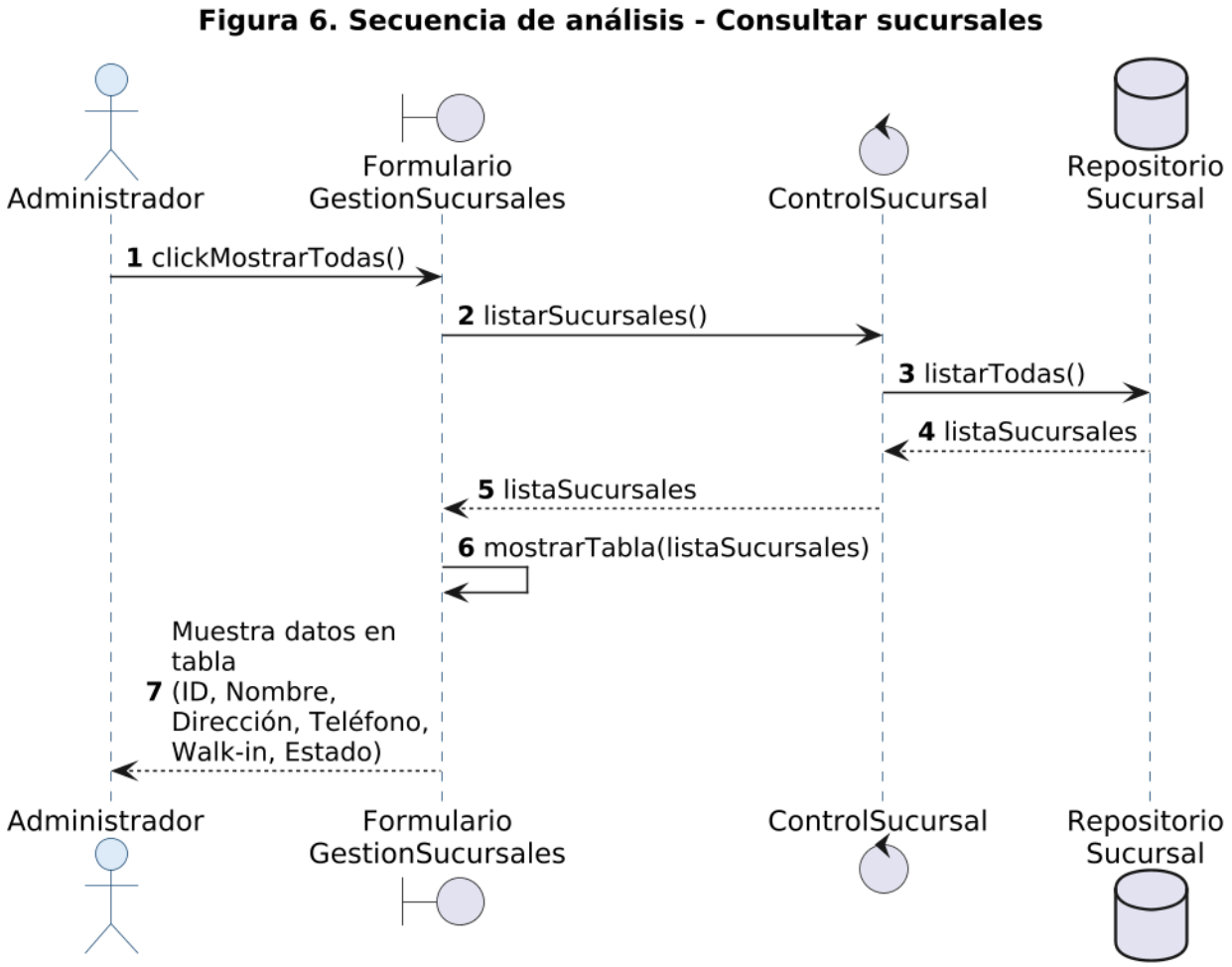


Figura 6. Secuencia de análisis – Consultar sucursales.

Código PlantUML

```
@startuml
skinparam shadowing false
skinparam dpi 150
skinparam maxMessageSize 130
skinparam ParticipantPadding 2
skinparam BoxPadding 4
skinparam defaultFontName Arial
skinparam sequence {
  ParticipantBackgroundColor #DCEBF7
  ParticipantBorderColor #1F4E79
  ActorBackgroundColor #DCEBF7
  ActorBorderColor #1F4E79
  LifeLineBorderColor #1F4E79
}
title Figura 6. Secuencia de análisis - Consultar sucursales
autonumber

actor "Administrador" as Adm
boundary "Formulario\nGestionSucursales" as UI
control "ControlSucursal" as Ctrl
database "Repositorio\nSucursal" as RepoS

Adm -> UI : clickMostrarTodas()
UI -> Ctrl : listarSucursales()
Ctrl -> RepoS : listarTodas()
RepoS --> Ctrl : listaSucursales
Ctrl --> UI : listaSucursales
UI -> UI : mostrarTabla(listaSucursales)
```

```
UI --> Adm : Muestra datos en tabla\n(ID, Nombre, Dirección, Teléfono, Walk-in, Estado)
@enduml
```

9. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Flujo alterno	Resultado esperado
Registrar sucursal	El Administrador está autenticado y el nombre no existe previamente.	Ingresar nombre, dirección, teléfono y walk-in → validar datos → verificar nombre único → guardar sucursal → registrar auditoría.	Datos inválidos o nombre duplicado: se rechaza el registro y se indica el campo.	Sucursal registrada como ACTIVA y visible en la tabla.
Modificar sucursal	La sucursal debe existir.	Seleccionar sucursal e ingresar datos nuevos → validar datos → buscar sucursal → verificar nombre → guardar cambios → registrar auditoría.	Datos inválidos o nombre usado por otra sucursal: se rechaza el cambio.	Datos de la sucursal actualizados.
Inactivar sucursal	La sucursal debe existir, estar activa y confirmarse la acción.	Seleccionar sucursal → confirmar → buscar sucursal → contar citas futuras → inactivar → registrar auditoría.	Tiene citas futuras: se bloquea con el mensaje de reasignación.	Sucursal INACTIVA, sin eliminarse y fuera de la búsqueda de disponibilidad.
Consultar sucursales	Pueden existir cero o más sucursales registradas.	Solicitar listado → recuperar registros → mostrar tabla.	No hay sucursales: se muestra la tabla vacía.	Tabla actualizada con ID, nombre, dirección, teléfono, walk-in y estado.

14. Conclusión técnica

En nuestra opinión, lo más valioso de este trabajo fue comprobar que los tres modelos no son piezas sueltas. Partimos de un requisito real del SRS, el RF-CFG-01, y lo fuimos aterrizando paso a paso: los casos de uso nos dijeron qué necesita hacer el Administrador con las sucursales, las clases de análisis nos mostraron qué objetos tienen que colaborar para lograrlo, y las secuencias nos confirmaron cómo se comunican esos objetos en el tiempo. Cuando una secuencia nos pedía un mensaje que no existía en el diagrama de clases, teníamos que volver y corregir la clase, y eso nos dio la seguridad de que el modelo quedó coherente.

Desde nuestra postura, el caso de Sucursales también nos obligó a pensar más allá del CRUD básico de la guía. La eliminación no podía ser un simple borrado porque el SRS exige baja lógica y bloquea la inactivación si hay citas futuras, así que apareció la clase RepositorioCita y la verificación como caso de uso incluido. Lo mismo pasó con la auditoría de RNF-13, que nos llevó a incorporar RegistroAuditoria. Estas decisiones salieron del análisis y no de la programación, que es justamente la idea de esta etapa.

Finalmente, la matriz de trazabilidad nos permite partir de cualquier requisito, como RF-CFG-01.3, y saber de inmediato en qué caso de uso se modela, qué clases intervienen, en qué figura está su secuencia y qué criterios de aceptación del SRS lo verifican. Consideramos que el mismo patrón se puede replicar sin mayor problema en los otros catálogos del módulo (consultorios, especialidades y médicos), y que lo trabajado aquí deja una base clara para pasar al diseño del sistema.